스킨케어 팀 · AI 서비스 프로젝트

설계서

시스템 설계 · API 명세 · DB 설계

과제명	스마트폰 촬영 영상 기반 피부 상태 정량화 및 맞춤 성분 추천 AI 서비스
팀	스킨케어 — 조장 김우주 / 박성근, 손승우, 이현승, 정도희
버전	v1.0 (2026-10-01)
원본	skincare-ai/docs/project/ (Markdown)

목차

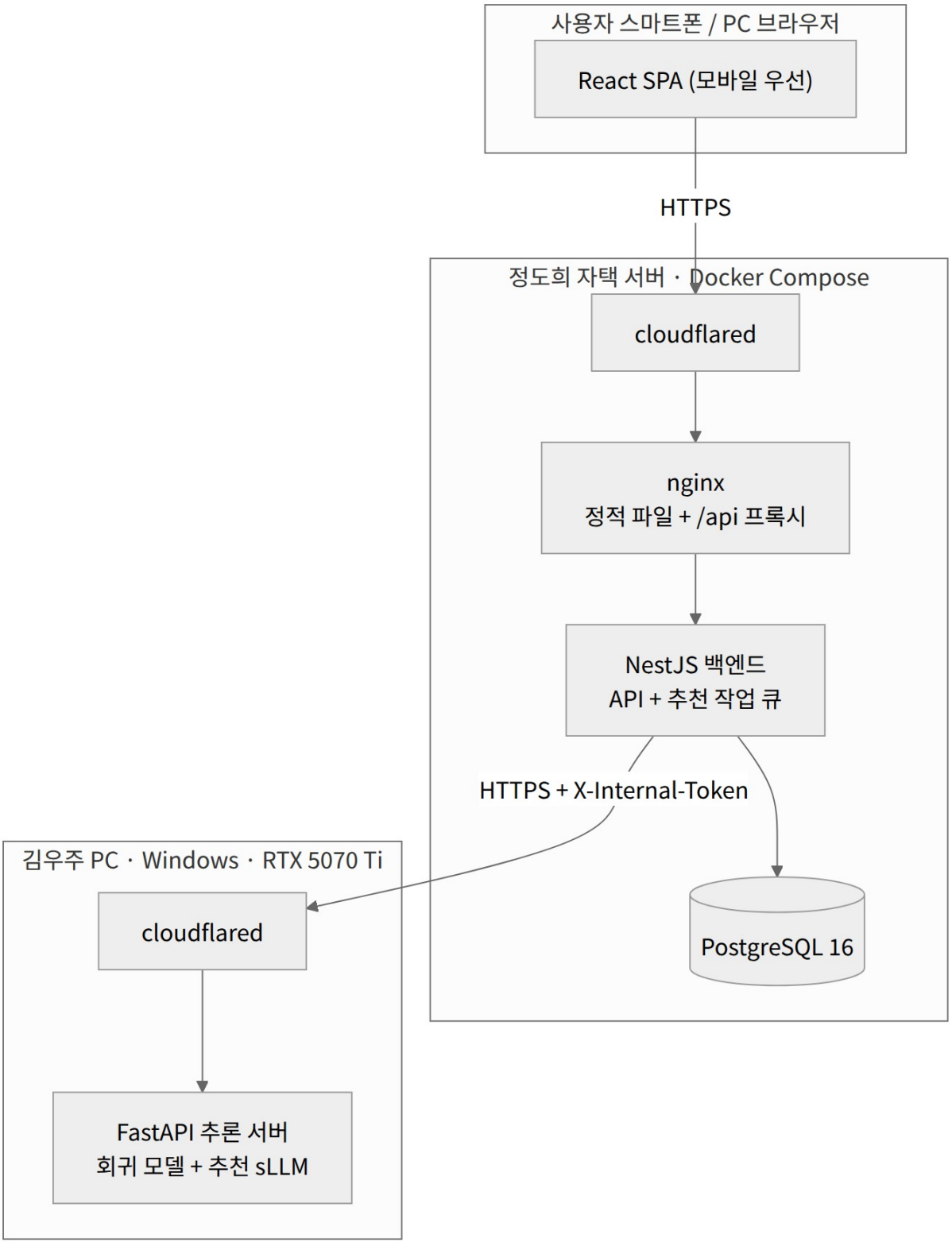
시스템 설계서

문서 번호 SC-02 · **버전** v1.0 (2026-10-01) · **작성** 김우주 · **관리** 손승우

1. 설계 원칙

1. **클라이언트는 AI 서버를 모른다.** 모든 요청은 백엔드를 거친다. AI 서버 주소· 토큰은 백엔드 환경변수에만 있다.
2. **측정은 빠르게, 추천은 줄 세워서.** 측정(약 3초)은 요청 안에서 끝내고, 추천(약 50초)은 백엔드 작업 큐가 GPU에 한 건씩 넣는다.
3. **얼굴 이미지는 디스크에 닿지 않는다.** 백엔드는 업로드를 메모리 버퍼로만 받아 AI 서버로 넘기고, 측정이 끝나면 버린다. 추천 단계에는 이미지가 필요 없다.
4. **AI 서버가 꺼져도 서비스는 산다.** AI 서버는 조장 PC에서만 돌기 때문에, 꺼져 있으면 분석만 막고 나머지 기능은 정상 동작한다.
5. **AI 출력은 그대로 믿지 않는다.** 금칙 표현 필터와 면책 문구는 AI 서버에서, 열거형 검증과 0–100 변환은 백엔드에서 한 번 더 한다.

2. 전체 구성

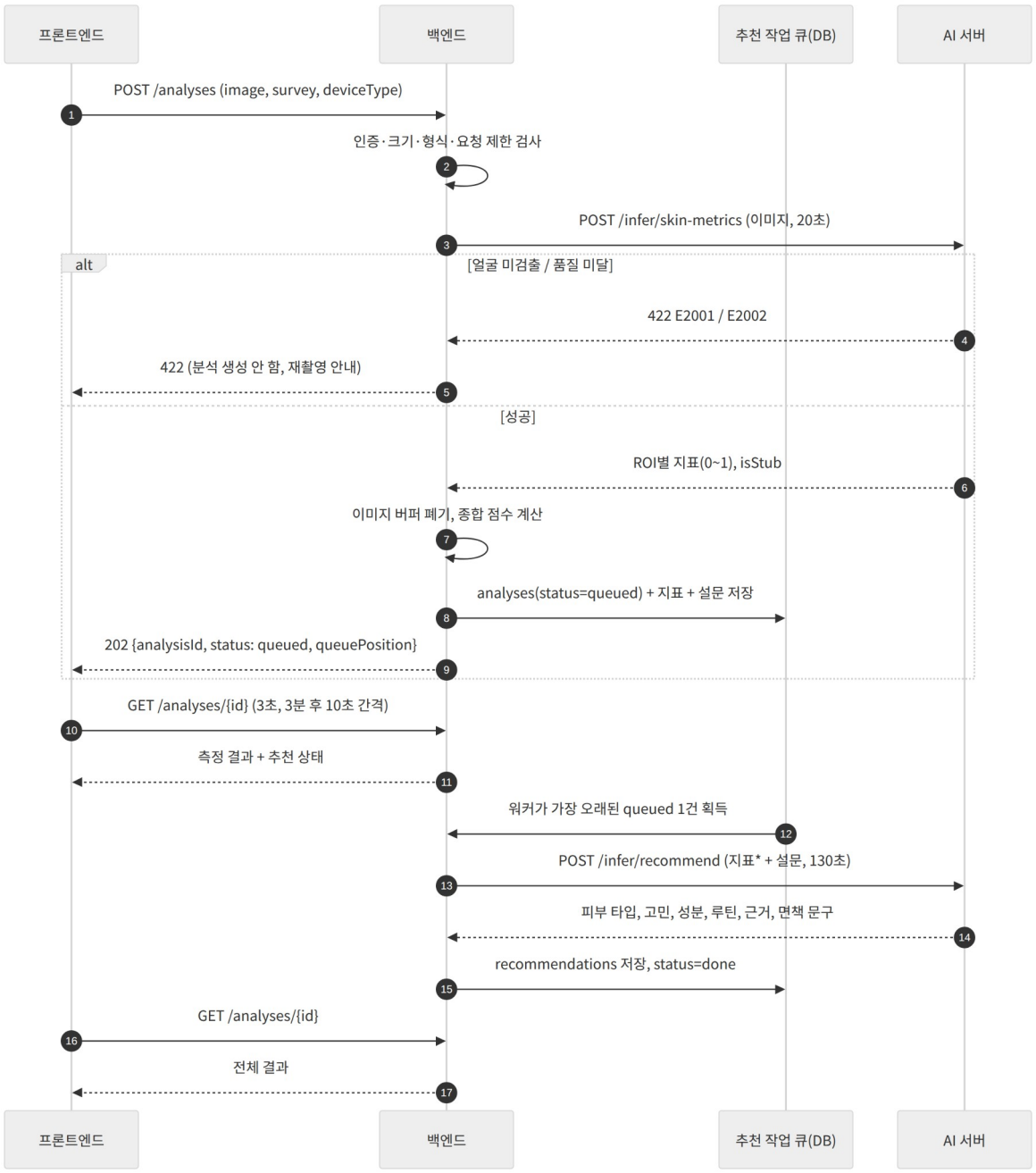


구성요소	기술	담당	책임
프론트엔드	React 18, TypeScript, Vite, React Router, TanStack Query	정도희	촬영· 업로드, 설문, 결과· 이력 화면, 상태 폴링

구성요소	기술	담당	책임
웹 서버	nginx	정도희	빌드 결과 서빙, `/api` → 백엔드 프록시, 업로드 크기 제한(12MB)
백엔드	NestJS 10, TypeORM, class-validator, Swagger	이현승	인증, 분석 API, 추천 작업 큐, AI 서버 호출, 결과 저장
DB	PostgreSQL 16	이현승	사용자, 분석, 지표, 설문, 추천 저장 (DB 설계서)
AI 추론 서버	FastAPI, PyTorch, transformers, PEFT	김우주	측정, 피부 타입· 고민 도출, 추천 생성, 금칙 표현 필터
터널	Cloudflare Tunnel	정도희, 김우주	포트포워딩 없이 HTTPS 외부 노출

3. 분석 처리 흐름

3.1 시퀀스

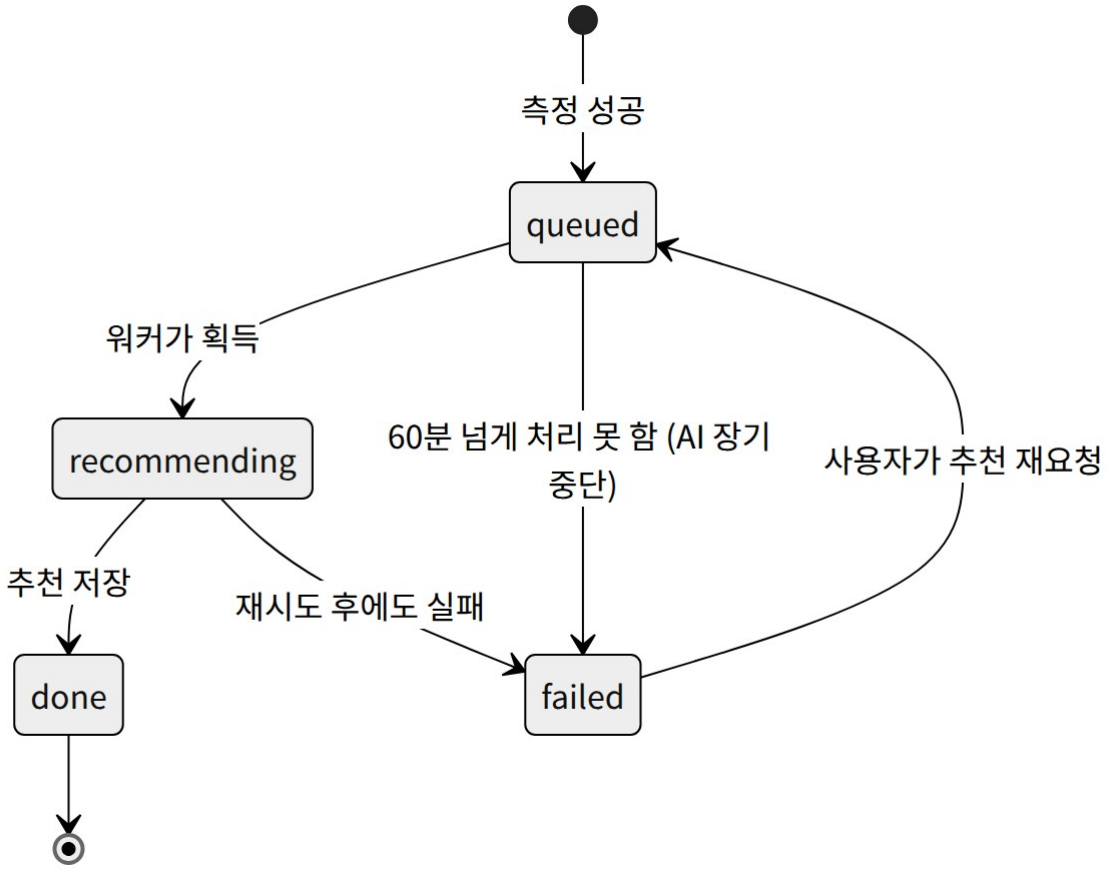


* 측정값이 데모 값(`is_demo`)이면 `metrics: null`로 보내, 가짜 값이 피부 타입· 고민 추정에 섞이지 않게 합니다. 이 경우 추천은 설문만으로 만들어집니다.

3.2 왜 측정과 추천을 나눴나

대안	문제
`/infer/analyze` 한 번 호출	사용자가 50초 이상 아무 결과도 못 봄. 대기 중 이미지를 백엔드가 계속 들고 있어야 함
측정· 추천 모두 큐	측정 대기 중 이미지를 임시 저장해야 함 → 개인정보 원칙 3 위반
측정은 요청 안에서, 추천만 큐 (채택)	측정 3초 만에 결과 화면 진입, 이미지는 요청이 끝나면 사라짐, GPU 생성은 순서대로

3.3 분석 상태 전이



측정 단계에서 실패하면 분석 행 자체를 만들지 않으므로 상태가 없습니다.

4. 백엔드 설계 (`skincare-backend`)

4.1 모듈

모듈	책임	주요 클래스
`auth`	가입, 로그인, 토큰 재발급· 폐기	`AuthController`, `AuthService`, `JwtStrategy`
`users`	내 정보 조회, 탈퇴	`UsersController`, `UsersService`
`analyses`	분석 생성· 조회· 목록· 추이· 삭제	`AnalysesController`, `AnalysesService`
`recommendation`	추천 작업 큐 워커	`RecommendationWorker`
`ai-client`	AI 서버 호출. `AI_MODE=real`이면 HTTP, `fixture`면 고정 응답	`AiClient`(인터페이스), `HttpAiClient`, `FixtureAiClient`
`system`	서비스 상태(AI 가용성, 대기열 길이)	`SystemController`
`common`	응답 포맷, 에러 필터, 요청 ID, 로깅	`ApiExceptionFilter`, `RequestIdMiddleware`

`FixtureAiClient`는 `skincare-ai`의 실제 응답 예시(JSON)를 그대로 돌려줍니다. 백엔드· 프론트 개발은 AI 서버가 꺼져 있어도 이 모드로 진행합니다.

4.2 추천 작업 큐

별도 큐 인프라(Redis 등) 없이 PostgreSQL을 큐로 씁니다. 백엔드 인스턴스는 1개입니다.

항목	설계
작업 획득	1초마다 `SELECT … FROM analyses WHERE status='queued' ORDER BY queued_at LIMIT 1 FOR UPDATE SKIP LOCKED` → `recommending`으로 변경
동시성	1 (`REC_WORKER_CONCURRENCY=1`). GPU 생성이 직렬이기 때문
호출 타임아웃	130초 (`AI_REC_TIMEOUT_MS`). AI 서버 내부 타임아웃(120초)보다 길게 둬서 보통은 AI의 504를 먼저 받음
재시도	**연결 오류와 503(E3003)만** 1회, 5초 후. **504(E3002)와 422는 재시도하지 않음** — AI 서버는 타임아웃이 나도 진행 중인 생성을 끝까지 돌리며 GPU를 붙잡고 있어, 곧바로 재시도하면 그 뒤에 줄을 서다 다시 타임아웃이 나고 다음 작업까지 밀린다
AI 서버 비가용	`aiAvailable=false`인 동안 작업 획득을 멈춤(대기열 유지). 복구되면 이어서 처리
장기 대기	`queued` 60분 초과 시 `failed`, `error_code=E3001` (AI 서버가 오래 꺼져 있던 경우)
서버 재시작 복구	기동 시 `recommending` 상태를 모두 `queued`로 되돌림 (`queued_at`은 유지해 순서 보존)
대기열 상한	`queued` 건수가 `REC_QUEUE_LIMIT`(10) 이상이면 새 분석 거부 (E3003). 10건 × 약 55초 ≈ 10분이 사용자가 기다릴 수 있는 한계로 봄
데모 측정값	`is_demo=true` 분석은 추천 요청의 `metrics`를 `null`로 보냄
대기 순번	내 `queued_at`보다 앞선 `queued` 건수 + 1
예상 대기	대기 순번 × 최근 완료 20건의 평균 추천 소요 시간 (없으면 55초)

4.3 AI 서버 에러 변환

AI 서버의 에러를 그대로 내보내지 않고 백엔드 코드로 바꿉니다. 특히 AI의 401을 그대로 전달하면 프론트가 "사용자 토큰 만료"로 오해해 로그아웃시킵니다.

AI 서버 응답	측정(`POST /analyses` 안)	추천(워커)
401 E1001 (내부 토큰 불일치)	503 E3001 + ERROR 로그	`failed` E3001 + ERROR 로그
413 E2003 / 415 E2004	그대로	—
422 E2001 / E2002	그대로 (재촬영 안내)	—
422 E4002	500 E4001 + ERROR 로그 (백엔드 변환 버그)	`failed` E4001
503 E3003	그대로	1회 재시도 후 `failed` E3003
504 E3002	그대로	재시도 없이 `failed` E3002
연결 실패· 응답 없음	503 E3001	1회 재시도 후 `failed` E3001

4.4 이미지 처리

- `multer` memoryStorage로 받고 디스크 저장 옵션을 쓰지 않는다.
- 형식 판별은 확장자가 아니라 파일 시그니처(JPEG `FF D8 FF`, PNG `89 50 4E 47`)로 한다.
- AI 서버 응답을 받은 즉시 버퍼 참조를 해제한다. 로그· 에러 메시지에 이미지 내용을 넣지 않는다.

4.5 종합 점수와 등급

AI 서버는 0–1 값을 주고, 백엔드는 그 값을 그대로 저장했다가 응답할 때 0–100 정수로 바꿉니다. 색소침착· 모공은 높을수록 나쁜 지표라 점수 계산 시 뒤집습니다.

```
value(m)     = round(원본 × 100)                        # 표시용 정수
score(m)     = moisture, elasticity  → value
               pigmentation, pore    → 100 − value
grade(score) = good(≥70) / normal(40–69) / care(<40)  # 정수 score 기준
overallScore = round( 평균( 원본 기반 score 4개, 반올림 전 값 ) )
ROI 점수     = 같은 식을 각 ROI에 적용 (byRoi[].overallScore, grade)
```

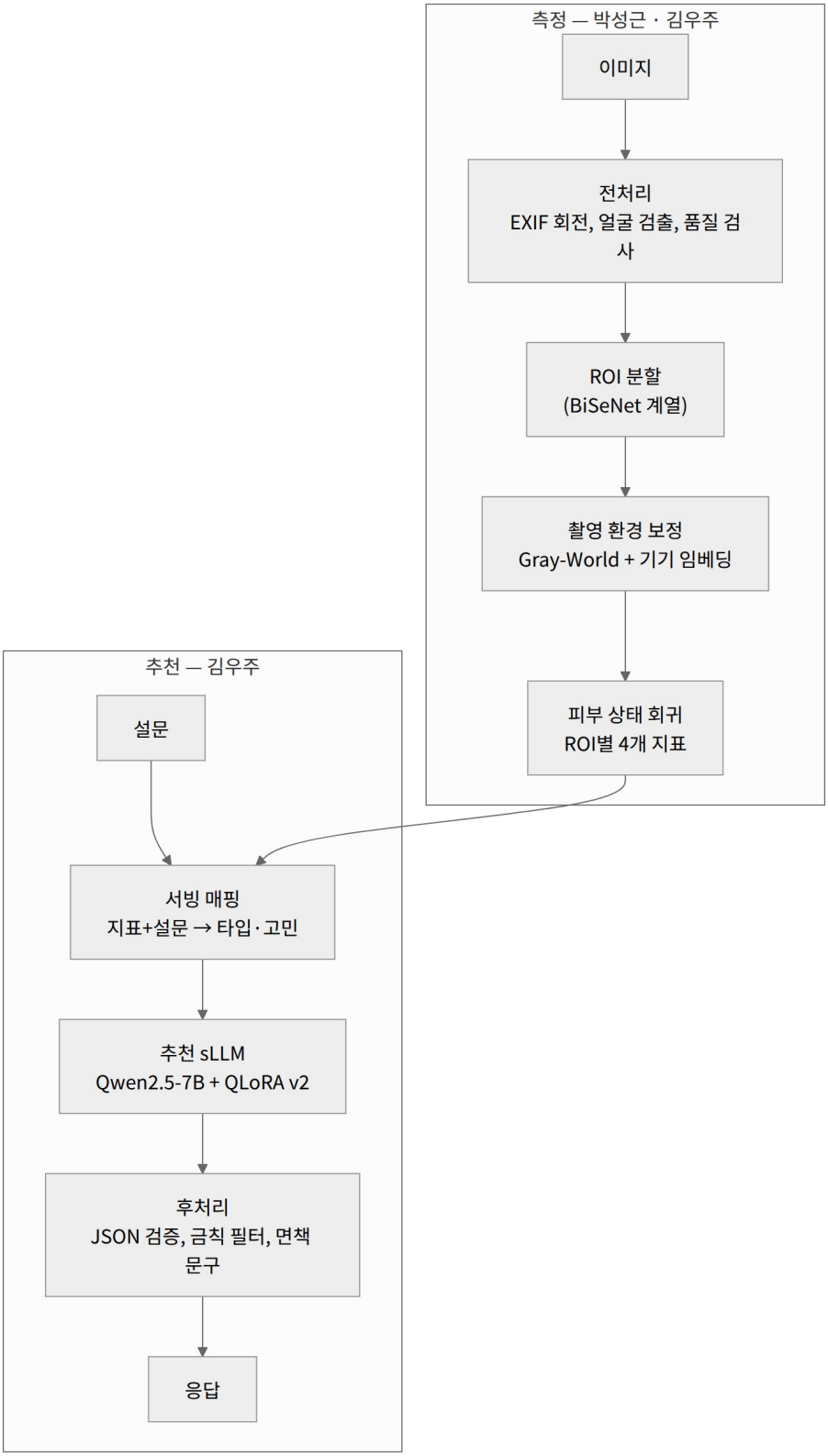
- `round`는 0.5 올림(JavaScript `Math.round`)입니다.
- 종합 점수는 `full_face` 기준입니다. `full_face`가 없으면 검출된 ROI 평균으로 계산합니다(AI 서버 어댑터와 같은 규칙).
- 등급은 백엔드가 계산해 내려주고, 프론트는 표시만 합니다(규칙 중복 방지).

5. 프론트엔드 설계 (`skincare-frontend`)

항목	설계
라우팅	React Router. 경로는 화면 설계서 §2
서버 상태	TanStack Query. 분석 상세는 `status`가 `queued`· `recommending`일 때만 refetch — 처음 3분은 3초, 이후 10초 간격, 15분이 지나면 중단
HTTP	axios 인스턴스 1개, `withCredentials: true`. 401(E1001) 응답 시 `/auth/refresh` 후 원 요청 재시도. **동시에 여러 요청이 401을 받아도 refresh는 한 번만 보내고**(single-flight) **나머지는 그 결과를 기다림** — 리프레시 토큰을 교체 방식으로 쓰기 때문에 두 번 보내면 재사용으로 판정돼 로그아웃됨
토큰	액세스 토큰은 메모리(상태)에만, 리프레시 토큰은 httpOnly 쿠키. **앱 시작 시 라우트 가드보다 먼저 `/auth/refresh`를 호출**해 로그인 상태를 복원
로컬 개발	Vite dev server proxy로 `/api` → `localhost:3000`. 같은 출처가 되어 쿠키· CORS 문제가 없음
타입	백엔드 Swagger(`/api/docs-json`)에서 `openapi-typescript`로 타입 생성
이미지	업로드 전 캔버스로 긴 변 1600px 리사이즈, JPEG 품질 0.9 (용량· 전송 시간 절감)

6. AI 서버 설계 (skincare-ai)

6.1 내부 파이프라인



단계	현재 상태 (10/1)	담당	코드 위치
전처리· ROI· 회귀· 보정	미연결 — stub이 데모 값 반환	박성근, 김우주	`server/skin_regressor.py`의 `SkinRegressor` 규약에 맞춰 연결
서빙 매핑	완료 (임계값 [확인필요])	김우주	`configs/serving_mapping.yaml`
추천 sLLM	완료 — v2 어댑터	김우주	`src/llm/`, `outputs/rec_sllm_v2/adapter`
금칙 필터	완료	김우주, 손승우 (규칙)	`configs/filter_rules.yaml`

6.2 동시성

작업	동시 처리	대기열	타임아웃(서버 내부)
측정	4	20	30초
추천	1	20	120초 (`REC_TIMEOUT_S`)

VRAM 사용량은 추천 sLLM(4-bit) 약 6GB + 생성 캐시, 회귀 모델 1GB 미만으로 16GB 안에 함께 올라갑니다. 학습 중에는 서버를 내려야 합니다.

6.3 폴백과 안전장치

- sLLM 출력이 JSON 형식을 어기면 1회 재생성, 다시 실패하면 규칙 기반 추천을 내보내고 `used_fallback: true`로 표시합니다.
- 금칙 필터가 문장을 지우거나 바꾸면 `filtered: true`. v2 기준 201건 중 1건(0.5%)에서만 개입했습니다.
- 응답 범위: `ingredients` 0–5개(`purpose`는 `null` 가능), `routine` 0개 이상, `reasoning` 3개 문단. 백엔드· 프론트는 빈 목록을 처리해야 합니다.
- 추천 모델은 **측정값 숫자나 부위를 직접 보지 않습니다.** 측정값은 서빙 매핑을 거쳐 피부 타입· 고민으로만 전달되므로, 추천 이유에 "이마 수분 61" 같은 수치는 나오지 않습니다.

7. 배포 구성

7.1 웹 서비스 (정도희 자택 서버)

```
docker-compose.yml
├─ web          nginx:1.27-alpine    (프론트 빌드 결과 + /api 프록시)
├─ backend      node:20-alpine       (NestJS, 포트 3000 — 외부 비공개)
├─ db           postgres:16-alpine   (볼륨 pgdata, 외부 비공개)
└─ cloudflared                       (app.<도메인> → web:80)
```

- 외부에 여는 포트는 없습니다. Cloudflare Tunnel이 HTTPS를 처리합니다.
- DB는 매일 03:00 `pg_dump`로 백업하고 7일치를 보관합니다.
- 배포: `main` 병합 → 서버에서 `git pull && docker compose up -d --build` (자동화는 S 우선순위).

7.2 AI 서버 (김우주 PC)

```
# 레포 루트에서
$env:REC_BACKEND="sllm"; .\scripts\run_server.ps1     # uvicorn :8000
cloudflared tunnel run skincare-ai                     # ai.<도메인> → localhost:8000
```

- 도메인이 없으면(결정 D3) `cloudflared tunnel --url http://localhost:8000`의 임시 주소를 쓰되, **재시작할 때마다 백엔드의 `AI_BASE_URL`을 바꿔야** 합니다.
- 학습· 평가를 돌리는 동안에는 추론 서버를 내리고 `GET /system/status`가 `aiAvailable: false`를 보여주게 둡니다.

8. 보안 · 개인정보 설계

위협 / 요구	대책	위치
비밀번호 유출	bcrypt cost 12, 로그인 실패 메시지를 이메일· 비밀번호 구분 없이 동일하게	백엔드
토큰 탈취	액세스 1시간, 리프레시 14일 httpOnly· Secure· SameSite=Strict 쿠키, 리프레시는 DB에 해시로 저장하고 사용 시 교체	백엔드
타인 분석 열람(IDOR)	모든 분석 조회에 `user_id` 조건. 없거나 남의 것이면 동일하게 404	백엔드
무차별 요청	로그인 IP당 분당 10회, 분석 사용자당 일 20건. 터널· nginx 뒤라 모든 요청의 출발 IP가 같으므로 **`CF-Connecting-IP` 헤더로 실제 IP를 판별**(nginx `real_ip_header`, Express `trust proxy`)	백엔드, nginx
AI 서버 직접 호출	내부 토큰(HMAC 비교, 미설정 시 전부 거부)	AI 서버
이미지 잔존	메모리 버퍼만 사용, 디스크 저장 경로 없음	백엔드, AI 서버
로그 유출	이미지· 설문 원문· 토큰 비기록. 이메일은 마스킹(`d***@gmail.com`)	전 구간
비밀값 노출	`.env`는 `.gitignore`, `.env.example`만 커밋. GitHub secret scanning 활성화	전 레포
학습 데이터 재배포	`data/`, `outputs/` git 제외	AI 레포
과도한 의료 표현	금칙 필터 + 면책 문구 + 화면 문구 규칙(화면 설계서 §4)	AI 서버, 프론트

9. 장애 대응

상황	감지	사용자에게 보이는 것	시스템 동작
AI 서버 꺼짐	백엔드가 30초마다 `/health` 확인	홈· 촬영 화면 "분석 서버 점검 중", 제출 버튼 비활성. 대기 중인 분석은 "서버가 돌아오면 이어서 작성돼요"	새 분석 거부(E3001), 워커 일시 정지, 대기열 유지
추천 타임아웃	AI 504 E3002	측정 결과 + "추천을 만들지 못했어요. 다시 시도"	재시도 없이 `failed`
백엔드 재시작	—	진행 중이던 추천이 잠시 멈춘 뒤 재개	`recommending` → `queued` 복구
대기열 가득	`queued` ⩾ 10	"지금 요청이 많아요. 잠시 후 다시 시도해 주세요"	503 E3003
DB 장애	헬스체크 실패	공통 오류 화면	500 E4001

10. 로깅 · 측정

- 모든 요청에 `X-Request-Id`(없으면 백엔드가 UUID 생성)를 붙이고, 백엔드→AI 서버 호출에도 그대로 전달합니다.
- `analyses` 테이블에 `measure_ms`, `recommend_ms`, `queue_wait_ms`를 저장합니다. 최종 성능 분석(응답 시간 분포, 대기 시간)은 이 값으로 계산합니다.
- AI 서버는 필터 개입· 폴백 발생을 INFO 로그로 남깁니다.

API 명세서

문서 번호 SC-03 · **버전** v1.0 (2026-10-01) · **작성** 김우주 · **검토** 이현승, 정도희

이 문서가 프론트 · 백엔드 · AI 서버 사이의 계약입니다. 바꿀 때는 협업 규칙 §5를 따릅니다. 백엔드 구현 후에는 Swagger(`/api/docs`)가 실제 동작 기준이 되며, 이 문서와 다르면 둘 중 하나를 고치는 PR을 올립니다.

1. 공통 규약

항목	규칙
Base URL	`https://app.<도메인>/api/v1` (로컬: `http://localhost:3000/api/v1`)
인증	`Authorization: Bearer <accessToken>`. 표의 "인증" 열이 `-`인 API만 예외
형식	요청 · 응답 `application/json; charset=utf-8` (업로드만 `multipart/form-data`)
필드 이름	camelCase. 열거형 **값**은 소문자 snake_case (`full_face`, `cheek_left`)
시간	ISO 8601 UTC (`2026-10-15T09:12:33Z`). 화면 표시는 프론트가 KST로 변환
ID	분석 ID는 ULID 26자 (`01J9Z3K5V8Q2M4N6P8R0S2T4V6`)
요청 추적	요청 헤더 `X-Request-Id`(선택). 응답에 항상 `X-Request-Id` 포함
페이지	`?page=1&size=20` (size 최대 50). 응답 `data.page` 객체 포함

1.1 응답 포맷

성공

```
{ "success": true, "data": { } }
```

실패

```
{
  "success": false,
  "error": { "code": "E2001", "message": "얼굴을 찾지 못했어요. 정면에서 다시 찍어 주세요.", "detail":
null }
}
```

- `message`는 **사용자에게 그대로 보여줘도 되는 문장**입니다. 프론트는 코드별 문구를 따로 만들지 않아도 됩니다.
- `detail`은 개발용(검증 실패 필드 목록 등). 화면에 보여주지 않습니다.
- `204 No Content`는 본문이 없습니다.

1.2 에러 코드

코드	HTTP	의미	발생	재시도	프론트 처리
E1001	401	토큰 없음 · 만료 · 위조	백엔드	토큰 재발급 후 1회	재발급 실패 시 로그인 화면
E1002	403	권한 없음	백엔드	안 함	공통 오류
E1003	401	이메일 또는 비밀번호 불일치	백엔드	안 함	로그인 폼 아래 문구
E1004	409	이미 가입된 이메일	백엔드	안 함	가입 폼 이메일 필드 오류

코드	HTTP	의미	발생	재시도	프론트 처리
E2001	422	얼굴 미검출	AI → 백엔드	안 함	재촬영 화면
E2002	422	이미지 품질 미달(흐림· 과노출· 저조도)	AI → 백엔드	안 함	재촬영 화면 + 사유
E2003	413	이미지 10MB 초과	백엔드(nginx 12MB), AI	안 함	업로드 전 차단이 원칙
E2004	415	JPEG· PNG가 아님	백엔드, AI	안 함	업로드 전 차단이 원칙
E3001	503	AI 서버 응답 없음· 연결 설정 오류, 또는 60분 넘게 처리되지 못함	백엔드	30초 후	"분석 서버 점검 중"
E3002	504	추론 타임아웃	백엔드, AI	추천 재요청 가능	측정 결과 + 추천 재시도 버튼
E3003	503	대기열 가득 참	백엔드, AI	1분 후	"요청이 많아요"
E4001	500	내부 오류	공통	안 함	공통 오류 (요청 ID 표시)
E4002	422	요청 형식 오류(검증 실패)	백엔드, AI	안 함	폼 필드 오류 (`detail` 참고)
E4003	404	리소스 없음 (남의 것 포함)	백엔드	안 함	"결과를 찾을 수 없어요"
E4004	429	요청 제한 초과 (진행 중 분석 1건, 일 20건(KST 기준), 로그인 분당 10회)	백엔드	`Retry-After` 헤더	안내 문구
E4005	409	지금 상태에서는 할 수 없는 요청 (예: 실패하지 않은 추천 재요청)	백엔드	안 함	화면 새로고침

AI 서버가 돌려준 에러는 백엔드가 이 표의 코드로 바꿔서 전달합니다(시스템 설계서 §4.3). 예를 들어 AI 서버의 401(내부 토큰 불일치)은 클라이언트에 503 E3001로 나갑니다.

2. 엔드포인트 목록 — 클라이언트 ↔ 백엔드

메서드	경로	설명	인증	요구사항
POST	`/auth/signup`	회원가입	-	FR-AUTH-01
POST	`/auth/login`	로그인	-	FR-AUTH-02
POST	`/auth/refresh`	액세스 토큰 재발급 (쿠키)	-	FR-AUTH-02
POST	`/auth/logout`	로그아웃	O	FR-AUTH-03
GET	`/users/me`	내 정보	O	—
DELETE	`/users/me`	회원 탈퇴	O	FR-AUTH-04
POST	`/analyses`	분석 요청 (측정 + 추천 대기열 등록)	O	FR-ANA-01, 02, 05
GET	`/analyses/{id}`	분석 결과· 진행 상태	O	FR-ANA-03, FR-REP-01, 02
POST	`/analyses/{id}/recommendation/retry`	실패한 추천 재요청	O	FR-ANA-04
GET	`/analyses`	이력 목록	O	FR-HIS-01
GET	`/analyses/{id}/trend`	직전 분석 대비 변화	O	FR-HIS-02
DELETE	`/analyses/{id}`	이력 삭제	O	FR-HIS-03
GET	`/system/status`	AI 가용성, 대기열	-	FR-SYS-01

3. 인증 · 사용자

3.1 POST /auth/signup

```
{
  "email": "user@example.com",
  "password": "Skin!2026",
  "nickname": "우주",
  "agreements": { "terms": true, "privacy": true, "imageProcessing": true, "over14": true }
}
```

필드	규칙
`email`	이메일 형식, 최대 255자, 소문자로 정규화
`password`	8–64자, 영문· 숫자· 특수문자 중 2종 이상
`nickname`	2–20자
`agreements.*`	4개 모두 `true`여야 함. 아니면 E4002

응답 `201`

```
{ "success": true, "data": { "userId": 42 } }
```

3.2 POST /auth/login

요청 `{ "email": "...", "password": "..." }`

응답 `200` + `Set-Cookie: refresh_token=…; HttpOnly; Secure; SameSite=Strict; Path=/api/v1/auth; Max-Age=1209600`

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOi…",
    "expiresIn": 3600,
    "user": { "id": 42, "email": "user@example.com", "nickname": "우주" }
  }
}
```

3.3 POST /auth/refresh

본문 없음. 쿠키의 리프레시 토큰을 검증하고 **새 리프레시 토큰으로 교체**합니다(재사용 감지 시 해당 사용자의 모든 리프레시 토큰 폐기). 응답은 로그인과 같은 구조.

3.4 POST /auth/logout → `204`

3.5 GET /users/me

```
{ "success": true, "data": { "id": 42, "email": "user@example.com", "nickname": "우주",
"createdAt": "2026-10-03T02:11:00Z", "analysisCount": 3 } }
```

3.6 DELETE /users/me → `204`

계정, 분석, 지표, 설문, 추천, 리프레시 토큰을 모두 즉시 삭제합니다. 진행 중인 추천 작업도 취소합니다.

4. 분석

4.1 POST /analyses

요청 `multipart/form-data`

필드	타입	필수	설명
`image`	file	O	JPEG/PNG, 10MB 이하, 짧은 변 720px 이상
`survey`	string(JSON)	O	아래 구조
`deviceType`	string	O	`smartphone` | `tablet` | `camera`

`survey`

```
{
  "gender": "female",
  "ageBand": "20s",
  "skinType": "combination",
  "sleepHours": 6,
  "waterIntakeLevel": "low",
  "sunExposureLevel": "high",
  "stressLevel": "medium",
  "concerns": ["pore", "pigmentation"],
  "currentProducts": ["토너", "수분크림"],
  "note": "여름에 번들거림이 심해요"
}
```

필드	필수	값
`gender`	O	`male` | `female`
`ageBand`	O	`10s` … `70s`
`skinType`	-	피부 타입 코드 (§6.3). 모르면 생략 또는 `null`
`sleepHours`	-	0–14
`waterIntakeLevel`, `sunExposureLevel`, `stressLevel`	-	`low` | `medium` | `high`
`concerns`	-	고민 코드 배열 (§6.4), 최대 3개
`currentProducts`	-	문자열 배열, 최대 5개, 각 30자
`note`	-	300자 이하

처리 순서: 인증 → 요청 제한(E4004) → 대기열 상한(E3003) → 형식 검증(E2003· E2004· E4002) → AI 측정(E2001· E2002· E3001) → 저장 → 응답

응답 `202 Accepted`

```
{
  "success": true,
  "data": {
    "analysisId": "01J9Z3K5V8Q2M4N6P8R0S2T4V6",
    "status": "queued",
    "queuePosition": 2,
    "estimatedWaitSec": 110
  }
}
```

측정이 실패하면 분석을 만들지 않고 `422 E2001/E2002` 또는 `503 E3001`을 바로 돌려줍니다.

4.2 GET /analyses/{id}

`status`가 `queued` · `recommending`이면 프론트는 다시 호출합니다 — 처음 3분은 3초, 이후 10초 간격, 15분이 지나면 중단.

응답 `200` (완료)


```
{
  "success": true,
  "data": {
    "analysisId": "01J9Z3K5V8Q2M4N6P8R0S2T4V6",
    "status": "done",
    "createdAt": "2026-10-15T09:12:33Z",
    "completedAt": "2026-10-15T09:13:31Z",
    "deviceType": "smartphone",
    "queuePosition": null,
    "estimatedWaitSec": null,
    "measurement": {
      "isDemo": false,
      "confidence": 0.87,
      "calibrationApplied": true,
      "overallScore": 63,
      "summary": [
        { "metric": "moisture",     "value": 61, "score": 61, "grade": "normal" },
        { "metric": "elasticity",   "value": 74, "score": 74, "grade": "good" },
        { "metric": "pigmentation", "value": 38, "score": 62, "grade": "normal" },
        { "metric": "pore",         "value": 45, "score": 55, "grade": "normal" }
      ],
      "byRoi": [
        { "roi": "forehead",   "moisture": 58, "elasticity": 71, "pigmentation": 35, "pore": 51,
"overallScore": 61, "grade": "normal" },
        { "roi": "cheek_left", "moisture": 66, "elasticity": 76, "pigmentation": 41, "pore": 38,
"overallScore": 66, "grade": "normal" }
      ],
      "modelVersion": "skin_reg_v1"
    },
    "recommendation": {
      "status": "done",
      "skinType": { "code": "combination", "label": "복합성" },
      "concerns": [
        { "code": "pore", "label": "모공" },
        { "code": "pigmentation", "label": "미백(색소침착/기미/칙칙함)" }
      ],
      "ingredients": [
        { "name": "나이아신아마이드", "purpose": "피지 분비와 모공 관리에 도움을 줄 수 있습니다.", "priority": 1
},
        { "name": "판테놀", "purpose": "피부 장벽을 지키는 데 도움을 줄 수 있습니다.", "priority": 2 },
        { "name": "알로에신", "purpose": "칙칙한 피부 톤 관리에 도움을 줄 수 있습니다.", "priority": 3 }
      ],
      "routine": [
        { "step": 1, "time": "morning", "action": "약산성 클렌저로 가볍게 세안합니다." },
        { "step": 2, "time": "morning", "action": "자외선 차단제를 바릅니다." },
        { "step": 3, "time": "evening", "action": "나이아신아마이드 세럼을 바릅니다." }
      ],
      "reasoning": [
        "복합성 피부로 T존의 피지 분비가 많아 모공이 도드라져 보일 수 있습니다. 자외선 노출이 많은 생활 습관은 피부 톤이
칙칙해지는 데 영향을 줄 수 있습니다. …",
        "나이아신아마이드는 피지 분비와 모공 관리에 도움을 줄 수 있습니다. 판테놀은 … 알로에신은 …",
        "아침에는 약산성 클렌저로 세안한 뒤 자외선 차단제를 바르고, 저녁에는 … 세럼을 사용하세요. …"
      ],
      "answer": "복합성 피부에 모공과 톤이 고민이라면 …",
      "inputNotes": ["나이는 연령대(20s) 중간값 25세로 추정"],
      "disclaimer": "본 결과는 참고용 정보이며 의학적 진단이나 치료를 대신하지 않습니다. 피부 이상이 지속되면 피부과
전문의와 상담하세요.",
      "filtered": false,
      "usedFallback": false,
      "modelVersion": "rec_sllm_v2"
    },
    "survey": {
      "gender": "female", "ageBand": "20s", "skinType": "combination", "sleepHours": 6,
      "waterIntakeLevel": "low", "sunExposureLevel": "high", "stressLevel": "medium",
      "concerns": ["pore", "pigmentation"], "currentProducts": ["토너", "수분크림"], "note": "여름에
번들거림이 심해요"
    },
    "error": null
```

```
  }
}
```

진행 중이면 `recommendation`은 `{ "status": "pending" }`만 있고, `queuePosition`· `estimatedWaitSec`이 채워집니다. 추천이 실패하면 `status: "failed"`, `recommendation: { "status": "failed" }`, `error: { "code": "E3002", "message": "…" }`입니다. 실패 코드는 E3001· E3002· E3003· E4001 중 하나입니다. **측정 결과는 실패와 무관하게 항상 포함**됩니다.

필드	설명
`measurement.isDemo`	`true`면 회귀 모델 연결 전 데모 값. 화면에 "데모 값" 배지 필수
`measurement.summary[].value`	원래 지표 값(0–100). 색소침착· 모공은 높을수록 나쁨
`measurement.summary[].score`	좋은 방향으로 통일한 값. 색소침착· 모공은 `100 − value`
`measurement.summary[].grade`	`score` 기준 등급 (§6.6)
`measurement.byRoi`	검출된 ROI만 포함. `full_face`는 `summary`에 있으므로 제외. 데모 값이면 빈 배열일 수 있음. `overallScore`· `grade`는 부위별 점 색에 사용
`recommendation.ingredients`	0–5개. `purpose`는 `null`일 수 있음
`recommendation.routine`	0개 이상. 비어 있으면 루틴 영역을 숨김
`recommendation.reasoning`	3개 문단 (문제 분석· 원인 / 성분 선택· 근거 / 사용법· 관리 방안). 각 문단은 여러 문장
`recommendation.answer`	모델이 쓴 요약 문단. 결과 화면 "한눈에 보기"에 사용
`recommendation.inputNotes`	추정해서 채운 입력 내역. "이렇게 가정했어요"로 접어서 표시
`recommendation.filtered`, `usedFallback`	화면 비표시, 로그· 분석용
`survey`	제출한 설문(클라이언트 코드). 다음 측정 때 설문 기본값으로 사용

4.3 POST /analyses/{id}/recommendation/retry

`status=failed`일 때만 허용. 아니면 `409 E4005`. 응답 `202`는 4.1과 같은 구조. 대기열 상한· 요청 제한도 4.1과 같이 적용합니다.

4.4 GET /analyses

`?page=1&size=20`

```
{
  "success": true,
  "data": {
    "items": [
      { "analysisId": "01J9…", "createdAt": "2026-10-15T09:12:33Z", "status": "done",
        "overallScore": 63, "isDemo": false, "concerns": ["pore", "pigmentation"] }
    ],
    "page": { "page": 1, "size": 20, "totalItems": 3, "totalPages": 1 }
  }
}
```

4.5 GET /analyses/{id}/trend

같은 사용자의 **직전 `done` 분석**과 비교합니다. 데모 값 분석끼리는 비교하지 않습니다.

```
{
  "success": true,
```

```
  "data": {
    "current":  { "analysisId": "01J9…", "date": "2026-10-15", "overallScore": 63,
                  "summary": { "moisture": 61, "elasticity": 74, "pigmentation": 38, "pore": 45 }
},
    "previous": { "analysisId": "01J8…", "date": "2026-09-28", "overallScore": 60,
                  "summary": { "moisture": 54, "elasticity": 72, "pigmentation": 40, "pore": 47 }
},
    "delta":    { "overallScore": 3, "moisture": 7, "elasticity": 2, "pigmentation": -2, "pore":
-2 }
  }
}
```

이전 분석이 없으면 `previous`, `delta`가 `null`입니다. `delta`는 원래 값의 차이이므로 색소침착· 모공은 **음수가 좋아진 것**입니다(화면 표시 시 주의).

4.6 DELETE /analyses/{id} → `204`

4.7 GET /system/status

```
{ "success": true, "data": { "aiAvailable": true, "queueLength": 3, "estimatedWaitSec": 165,
"checkedAt": "2026-10-15T09:12:00Z" } }
```

백엔드가 30초마다 AI `/health`를 확인한 결과입니다. `aiAvailable=false`면 프론트는 분석 제출을 막습니다.

5. 백엔드 ↔ AI 서버

항목	값
Base URL	환경변수 `AI_BASE_URL` (예: `https://ai.<도메인>`)
인증	`X-Internal-Token: <AI_INTERNAL_TOKEN>`. `/health`만 예외
필드 이름	응답은 snake_case(Python 관례). **백엔드가 camelCase로 변환해 저장· 응답**
응답 형식	성공은 본문 그대로(`{success, data}` 감싸지 않음), 실패는 §1.1과 같은 `{success: false, error}`
실제 스키마	`skincare-ai/server/schemas.py`, 서버 실행 후 `http://localhost:8000/docs`

메서드	경로	백엔드 사용	타임아웃	재시도
GET	`/health`	30초마다 상태 확인	5초	—
GET	`/model-info`	기동 시 1회 로그	5초	—
POST	`/infer/skin-metrics`	`POST /analyses` 안에서 동기 호출	20초	연결 오류만 1회
POST	`/infer/recommend`	추천 워커에서 호출	130초	연결 오류· 503만 1회. 504**는 재시도 금지** (이전 생성이 GPU를 계속 점유)
POST	`/infer/analyze`	사용 안 함 (테스트용)	—	—

5.1 POST /infer/skin-metrics

요청 `multipart/form-data`: `image`(file), `device_type`(`smartphone`|`tablet`|`camera`)

응답 `200`

```
{
  "roi_detected": ["full_face", "forehead", "glabella", "cheek_left", "cheek_right", "chin"],
  "metrics": {
```

```
    "full_face": { "moisture": 0.61, "elasticity": 0.74, "pigmentation": 0.38, "pore": 0.45 },
    "forehead":  { "moisture": 0.58, "elasticity": 0.71, "pigmentation": 0.35, "pore": 0.52 }
  },
  "confidence": 0.87,
  "calibration_applied": true,
  "is_stub": false,
  "model_version": "skin_reg_v1",
  "inference_ms": 840
}
```

- 값은 0–1. 백엔드는 0–1 **원본을 그대로 저장**하고, 클라이언트에 응답할 때 `round(v × 100)`으로 바꿉니다.
- `is_stub: true`(현재 기본)면 해시로 만든 가짜 값입니다. 백엔드는 `analyses.is_demo=true`로 저장합니다.
- 에러: `401 E1001`, `413 E2003`, `415 E2004`, `422 E2001/E2002`(회귀 모델 연결 후), `503 E3003`, `504 E3002`

5.2 POST /infer/recommend

요청

```
{
  "metrics": { "full_face": { "moisture": 0.61, "elasticity": 0.74, "pigmentation": 0.38, "pore":
0.45 } },
  "survey": {
    "gender": "female",
    "ageBand": "20s",
    "skinType": "복합성",
    "sleepHours": 6,
    "waterIntakeLevel": "low",
    "sunExposureLevel": "high",
    "stressLevel": "medium",
    "currentProducts": ["토너", "수분크림"],
    "selfReportedConcerns": ["모공", "미백(색소침착/기미/칙칙함)"],
    "note": "여름에 번들거림이 심해요"
  }
}
```

측정값이 데모 값(`is_demo`)인 분석은 `"metrics": null`로 보냅니다. 그러면 AI 서버는 설문만으로 피부 타입· 고민을 정합니다.

백엔드 변환 규칙 (클라이언트 설문 → AI 설문)

클라이언트	AI 서버	변환
`skinType: "combination"`	`skinType: "복합성"`	§6.3 코드 → 라벨 (영문 코드도 인식하지만 라벨로 통일)
`concerns: ["pore"]`	`selfReportedConcerns: ["모공"]`	§6.4 코드 → 라벨. **코드를 그대로 보내면 인식되지 않음**
(저장된 측정값)	`metrics` (0–1)	DB에 저장한 0–1 원본을 ROI별로 그대로 전달
나머지	같은 이름	그대로

응답 `200`

```
{
  "skin_type": "복합성",
  "concerns": ["모공", "미백(색소침착/기미/칙칙함)"],
  "ingredients": [{ "name": "나이아신아마이드", "purpose": "…", "priority": 1 }],
  "routine": [{ "step": 1, "time": "morning", "action": "약산성 클렌저로 가볍게 세안합니다." }],
  "reasoning": ["(문제 분석·원인 문단)", "(성분 선택·근거 문단)", "(사용법·관리 방안 문단)"],
  "answer": "…",
  "disclaimer": "본 결과는 참고용 정보이며 …",
  "filtered": false,
  "used_fallback": false,
  "input_notes": ["나이는 연령대(20s) 중간값 25세로 추정"],
  "model_version": "rec_sllm_v2",
```

```
  "inference_ms": 48100
}
```

- `skin_type`, `concerns`는 **한글 라벨**로 옵니다. 백엔드가 §6.3· 6.4로 코드를 붙여 저장합니다. 표에 없는 값이 오면 E4001로 처리하고 로그를 남깁니다.
- `ingredients`는 0–5개(`purpose`는 `null` 가능), `routine`은 0개 이상, `routine[].time`은 §6.5. 서버에 성분 사전(`ingredient_dict.json`)이 없으면 `ingredients`가 빈 배열입니다.
- 추천 모델은 측정 수치· 부위를 직접 보지 않으므로 `reasoning`에 수치가 나오지 않습니다.
- 에러: `401 E1001`, `422 E4002`, `503 E3003`, `504 E3002`

5.3 GET /health

```
{ "status": "ok", "rec_loaded": true, "skin_loaded": true, "rec_backend": "sllm", "skin_backend":
"stub" }
```

`status`가 `ok`가 아니거나 5초 안에 응답이 없으면 `aiAvailable=false`.

6. 열거형 (enum)

6.1 상태

필드	값
분석 `status`	`queued`(추천 대기) · `recommending`(추천 생성 중) · `done` · `failed`
`recommendation.status`	`pending`(queued· recommending일 때) · `done` · `failed`

6.2 ROI `roi`

`full_face`, `forehead`, `glabella`(미간), `eye_left`, `eye_right`, `cheek_left`, `cheek_right`, `lip`, `chin`

6.3 피부 타입

코드	라벨 (AI 서버 값)
`dry`	건성
`normal`	중성
`oily`	지성
`combination`	복합성

6.4 피부 고민 (8종)

코드	라벨 (AI 서버 값)	화면 표기
`pigmentation`	미백(색소침착/기미/칙칙함)	피부 톤· 잡티
`pore`	모공	모공
`wrinkle`	주름	주름
`acne`	여드름/뾰루지	트러블

코드	라벨 (AI 서버 값)	화면 표기
redness	붉어짐(홍조)	붉은기
dryness	과각질/악건성	건조· 각질
sagging	피부처짐/탄력저하	탄력
sensitivity	민감성(트러블/자극감)	민감함

"화면 표기"는 의료 용어로 읽히지 않게 고른 사용자용 표현입니다. acne 라벨의 "여드름"은 질환명이므로 화면에는 "트러블"로 씁니다.

6.5 루틴 시간 time

morning · evening · daily · weekly · anytime

6.6 등급 grade

코드	score 범위	화면 표기
good	70–100	양호
normal	40–69	보통
care	0–39	관리 필요

6.7 기타

deviceType: smartphone · tablet · camera / gender: male · female / ageBand: 10s ⋯ 70s / 수준: low · medium · high

7. 개발용 고정 응답 (fixture)

AI_MODE=fixture일 때 백엔드는 AI 서버 대신 test/fixtures/ai/ 아래 JSON을 돌려줍니다. 어떤 응답을 줄지는 환경변수 AI_FIXTURE_SCENARIO로 고르고, **운영(NODE_ENV=production)이 아닐 때만** 요청 헤더 X-Fixture-Scenario로 요청마다 바꿀 수 있습니다(테스트용).

시나리오	측정	추천
ok (기본)	skin-metrics.ok.json (§5.1, is_stub: true)	recommend.ok.json (§5.2), 1초 지연
no-face	422 E2001	—
rec-timeout	정상	504 E3002 즉시 반환
rec-slow	정상	60초 지연 후 정상 (대기열 테스트용)
ai-down	연결 실패	연결 실패

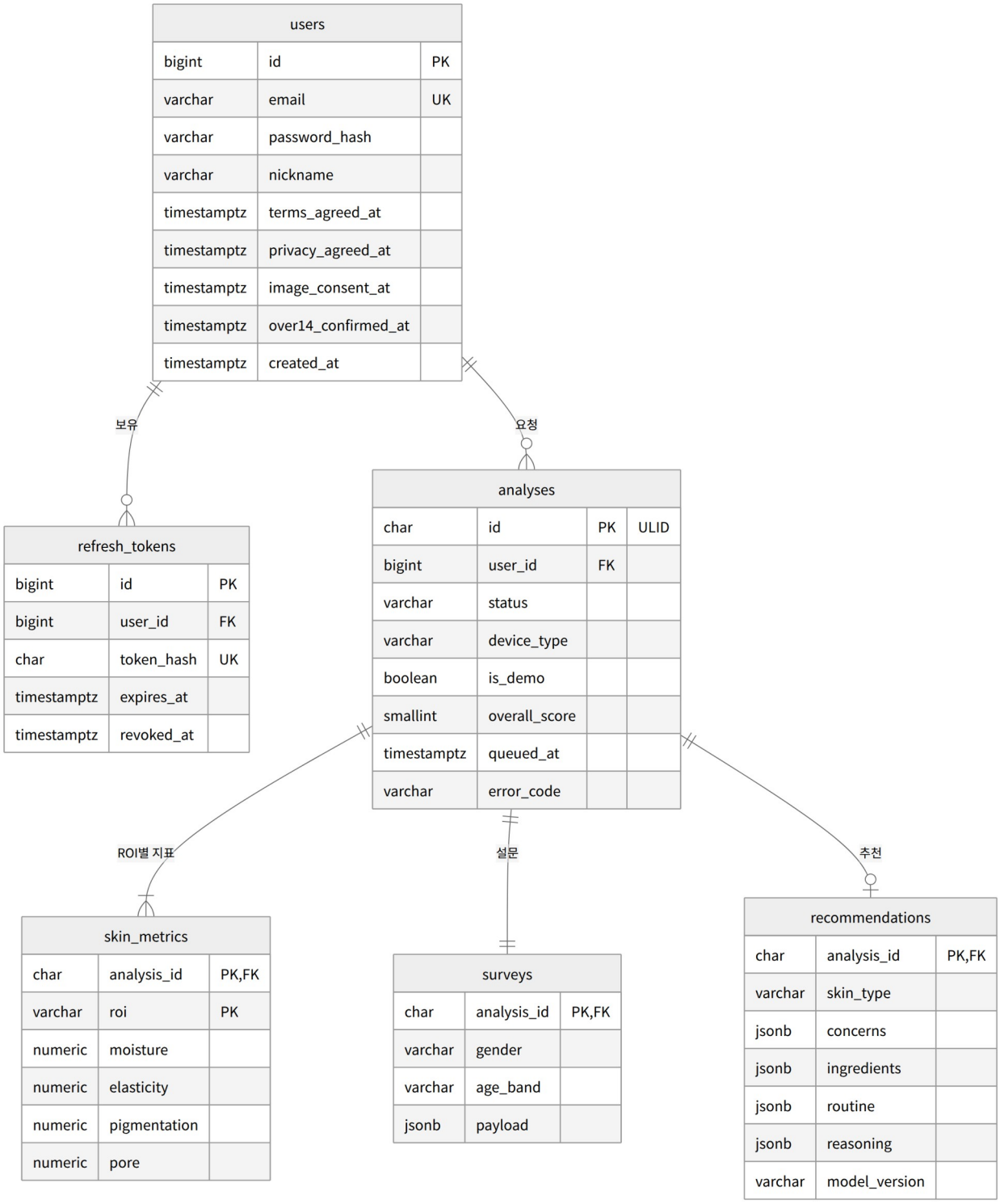
recommend.ok.json은 김우주가 v2 모델 실제 출력으로 만들어 10/3까지 skincare-ai/docs/fixtures/에 올립니다.

DB 설계서

문서 번호 SC-04 · **버전** v1.0 (2026-10-01) · **작성** 김우주 · **구현** 이현승

PostgreSQL 16, 스키마 `public`, 문자 인코딩 UTF-8, 시간대 UTC.

1. ERD



2. 테이블 정의

2.1 users — 회원

컬럼	타입	NULL	기본값	설명
id	BIGSERIAL	N		PK
email	VARCHAR(255)	N		소문자 정규화, UNIQUE
password_hash	VARCHAR(100)	N		bcrypt (cost 12)
nickname	VARCHAR(20)	N		
terms_agreed_at	TIMESTAMPTZ	N		이용약관 동의 시각
privacy_agreed_at	TIMESTAMPTZ	N		개인정보 수집· 이용 동의 시각
image_consent_at	TIMESTAMPTZ	N		얼굴 이미지 처리 동의 시각
over14_confirmed_at	TIMESTAMPTZ	N		만 14세 이상 확인 시각
created_at	TIMESTAMPTZ	N	now()	
updated_at	TIMESTAMPTZ	N	now()	

2.2 refresh_tokens — 리프레시 토큰

컬럼	타입	NULL	기본값	설명
id	BIGSERIAL	N		PK
user_id	BIGINT	N		FK users, ON DELETE CASCADE
token_hash	CHAR(64)	N		SHA-256(토큰). 원문 저장 금지. UNIQUE
expires_at	TIMESTAMPTZ	N		발급 + 14일
revoked_at	TIMESTAMPTZ	Y		로그아웃· 교체 시각
replaced_by	BIGINT	Y		교체한 새 토큰 id (재사용 감지용)
created_at	TIMESTAMPTZ	N	now()	

2.3 analyses — 분석

컬럼	타입	NULL	기본값	설명
id	CHAR(26)	N		ULID. URL에 노출
user_id	BIGINT	N		FK users, ON DELETE CASCADE
status	VARCHAR(16)	N	'queued'	`queued` · `recommending` · `done` · `failed`
device_type	VARCHAR(16)	N		`smartphone` · `tablet` · `camera`
is_demo	BOOLEAN	N		AI `is_stub` 그대로. true면 데모 측정값
confidence	NUMERIC(4,3)	N		측정 신뢰도 0–1
calibration_applied	BOOLEAN	N	false	촬영 환경 보정 적용 여부
overall_score	SMALLINT	N		0–100 (시스템 설계서 §4.5)
skin_model_version	VARCHAR(40)	N		예: `skin_reg_v1`

컬럼	타입	NULL	기본값	설명
measure_ms	INTEGER	N		측정 API 왕복 시간
queued_at	TIMESTAMPTZ	N	now()	큐 진입 (재요청 시 갱신)
rec_started_at	TIMESTAMPTZ	Y		워커 획득 시각
completed_at	TIMESTAMPTZ	Y		`done` · `failed` 확정 시각
queue_wait_ms	INTEGER	Y		rec_started_at − queued_at
recommend_ms	INTEGER	Y		추천 API 왕복 시간
attempt_count	SMALLINT	N	0	추천 호출 횟수 (재시도 포함)
error_code	VARCHAR(10)	Y		실패 시 에러 코드
created_at	TIMESTAMPTZ	N	now()	

2.4 skin_metrics — ROI별 지표

컬럼	타입	NULL	설명
analysis_id	CHAR(26)	N	PK, FK analyses, ON DELETE CASCADE
roi	VARCHAR(16)	N	PK. API §6.2
moisture	NUMERIC(5,4)	N	0–1 원본
elasticity	NUMERIC(5,4)	N	0–1 원본
pigmentation	NUMERIC(5,4)	N	0–1 원본 (높을수록 나쁨)
pore	NUMERIC(5,4)	N	0–1 원본 (높을수록 나쁨)

0–1 원본을 저장하는 이유: 추천 API에 그대로 다시 넘기고, 최종 성능 분석에서 반올림 손실 없이 쓰기 위해서입니다. 0–100 변환은 응답 시점에 합니다.

> TypeORM은 `NUMERIC` 컬럼을 **문자열**로 돌려줍니다. 점수 계산이 문자열 연결로 깨지지 않도록 엔티티에 `transformer: { to: (v) => v, from: (v) => (v === null ? null : Number(v)) }`를 지정합니다(`confidence` 포함).

2.5 surveys — 설문

컬럼	타입	NULL	설명
analysis_id	CHAR(26)	N	PK, FK analyses, ON DELETE CASCADE
gender	VARCHAR(8)	N	통계 · 필터용으로 분리
age_band	VARCHAR(4)	N	〃
skin_type	VARCHAR(16)	Y	사용자 자가응답 코드
payload	JSONB	N	제출한 설문 전체 (API §4.1)

2.6 recommendations — 추천

컬럼	타입	NULL	설명
analysis_id	CHAR(26)	N	PK, FK analyses, ON DELETE CASCADE
skin_type	VARCHAR(16)	N	도출된 피부 타입 **코드**
concerns	JSONB	N	고민 코드 배열 `["pore","pigmentation"]`

컬럼	타입	NULL	설명
ingredients	JSONB	N	`[{name, purpose, priority}]`
routine	JSONB	N	`[{step, time, action}]`
reasoning	JSONB	N	문자열 배열
answer	TEXT	N	요약 문단
disclaimer	TEXT	N	면책 문구
input_notes	JSONB	N	추정 입력 내역
filtered	BOOLEAN	N	금칙 필터 개입 여부
used_fallback	BOOLEAN	N	규칙 기반 폴백 여부
model_version	VARCHAR(40)	N	예: `rec_sllm_v2`
inference_ms	INTEGER	N	AI 서버 내부 생성 시간
created_at	TIMESTAMPTZ	N	now()

3. DDL

```
CREATE TABLE users (
  id                  BIGSERIAL PRIMARY KEY,
  email               VARCHAR(255) NOT NULL UNIQUE,
  password_hash       VARCHAR(100) NOT NULL,
  nickname            VARCHAR(20)  NOT NULL,
  terms_agreed_at     TIMESTAMPTZ  NOT NULL,
  privacy_agreed_at   TIMESTAMPTZ  NOT NULL,
  image_consent_at    TIMESTAMPTZ  NOT NULL,
  over14_confirmed_at TIMESTAMPTZ  NOT NULL,
  created_at          TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at          TIMESTAMPTZ  NOT NULL DEFAULT now()
);

CREATE TABLE refresh_tokens (
  id          BIGSERIAL PRIMARY KEY,
  user_id     BIGINT      NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  token_hash  CHAR(64)    NOT NULL UNIQUE,
  expires_at  TIMESTAMPTZ NOT NULL,
  revoked_at  TIMESTAMPTZ,
  replaced_by BIGINT,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX idx_refresh_tokens_user ON refresh_tokens(user_id);

CREATE TABLE analyses (
  id                  CHAR(26)     PRIMARY KEY,
  user_id             BIGINT       NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status              VARCHAR(16)  NOT NULL DEFAULT 'queued'
                      CHECK (status IN ('queued','recommending','done','failed')),
  device_type         VARCHAR(16)  NOT NULL
                      CHECK (device_type IN ('smartphone','tablet','camera')),
  is_demo             BOOLEAN      NOT NULL,
  confidence          NUMERIC(4,3) NOT NULL CHECK (confidence BETWEEN 0 AND 1),
  calibration_applied BOOLEAN      NOT NULL DEFAULT false,
  overall_score       SMALLINT     NOT NULL CHECK (overall_score BETWEEN 0 AND 100),
  skin_model_version  VARCHAR(40)  NOT NULL,
  measure_ms          INTEGER      NOT NULL,
  queued_at           TIMESTAMPTZ  NOT NULL DEFAULT now(),
  rec_started_at      TIMESTAMPTZ,
  completed_at        TIMESTAMPTZ,
  queue_wait_ms       INTEGER,
  recommend_ms        INTEGER,
  attempt_count       SMALLINT     NOT NULL DEFAULT 0,
  error_code          VARCHAR(10),
  created_at          TIMESTAMPTZ  NOT NULL DEFAULT now()
);
```

```
CREATE INDEX idx_analyses_user_created ON analyses(user_id, created_at DESC);
CREATE INDEX idx_analyses_queue ON analyses(queued_at) WHERE status = 'queued';

CREATE TABLE skin_metrics (
  analysis_id  CHAR(26)     NOT NULL REFERENCES analyses(id) ON DELETE CASCADE,
  roi          VARCHAR(16)  NOT NULL,
  moisture     NUMERIC(5,4) NOT NULL CHECK (moisture     BETWEEN 0 AND 1),
  elasticity   NUMERIC(5,4) NOT NULL CHECK (elasticity   BETWEEN 0 AND 1),
  pigmentation NUMERIC(5,4) NOT NULL CHECK (pigmentation BETWEEN 0 AND 1),
  pore         NUMERIC(5,4) NOT NULL CHECK (pore         BETWEEN 0 AND 1),
  PRIMARY KEY (analysis_id, roi)
);

CREATE TABLE surveys (
  analysis_id CHAR(26)    PRIMARY KEY REFERENCES analyses(id) ON DELETE CASCADE,
  gender      VARCHAR(8)  NOT NULL,
  age_band    VARCHAR(4)  NOT NULL,
  skin_type   VARCHAR(16),
  payload     JSONB       NOT NULL
);

CREATE TABLE recommendations (
  analysis_id   CHAR(26)    PRIMARY KEY REFERENCES analyses(id) ON DELETE CASCADE,
  skin_type     VARCHAR(16) NOT NULL,
  concerns      JSONB       NOT NULL,
  ingredients   JSONB       NOT NULL,
  routine       JSONB       NOT NULL,
  reasoning     JSONB       NOT NULL,
  answer        TEXT        NOT NULL,
  disclaimer    TEXT        NOT NULL,
  input_notes   JSONB       NOT NULL DEFAULT '[]',
  filtered      BOOLEAN     NOT NULL,
  used_fallback BOOLEAN     NOT NULL,
  model_version VARCHAR(40) NOT NULL,
  inference_ms  INTEGER     NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

4. 주요 쿼리

추천 작업 획득 (워커, 1초마다)

```
UPDATE analyses SET status = 'recommending', rec_started_at = now(),
       queue_wait_ms = (EXTRACT(EPOCH FROM now() - queued_at) * 1000)::int,
       attempt_count = attempt_count + 1
 WHERE id = (SELECT id FROM analyses WHERE status = 'queued'
             ORDER BY queued_at LIMIT 1 FOR UPDATE SKIP LOCKED)
RETURNING id;
```

대기 순번

```
SELECT COUNT(*) + 1 FROM analyses
 WHERE status = 'queued' AND queued_at < (SELECT queued_at FROM analyses WHERE id = $1);
```

평균 추천 시간 (예상 대기 계산)

```
SELECT COALESCE(AVG(recommend_ms), 55000) FROM
  (SELECT recommend_ms FROM analyses WHERE status = 'done' AND recommend_ms IS NOT NULL
    ORDER BY completed_at DESC LIMIT 20) t;
```

서버 기동 시 복구

```
UPDATE analyses SET status = 'queued' WHERE status = 'recommending';
```

장기 대기 처리 (1분마다)

```
UPDATE analyses SET status = 'failed', error_code = 'E3001', completed_at = now()
 WHERE status = 'queued' AND queued_at < now() - interval '60 minutes';
```

대기열 상한 확인 (분석 생성 전)

```
SELECT COUNT(*) FROM analyses WHERE status = 'queued';   -- REC_QUEUE_LIMIT(10) 이상이면 E3003
```

AI 서버가 비가용(`aiAvailable=false`)인 동안 워커는 작업 획득 쿼리를 실행하지 않습니다.

5. 데이터 보존 · 삭제

데이터	저장 위치	보존	삭제
얼굴 원본 이미지	**저장하지 않음** (요청 중 메모리)	—	측정 응답 직후
계정	users	탈퇴 시까지	탈퇴 즉시 (CASCADE로 하위 전부)
분석· 지표· 설문· 추천	analyses 외 4개	사용자가 삭제하거나 탈퇴할 때까지	즉시
리프레시 토큰	refresh_tokens	만료 후 7일	매일 03:30 배치
DB 백업	서버 로컬 디스크	7일	순환 삭제
애플리케이션 로그	컨테이너 로그	14일	Docker 로그 로테이션

탈퇴한 사용자의 데이터는 백업에 최대 7일 남을 수 있습니다. 개인정보 처리방침에 이 사실을 적습니다(담당 손승우).

6. 마이그레이션 규칙

- TypeORM 마이그레이션만 사용합니다. 운영 DB에서 `synchronize: true` 금지.
- 파일 이름: `{타임스탬프}-{동사}{대상}.ts` (예: `1727740800000-CreateAnalyses.ts`).
- 스키마를 바꾸는 PR에는 마이그레이션 파일과 이 문서 수정이 함께 들어가야 합니다.
- 개발· 테스트용 시드(`npm run seed`): `test01`~`test40@skincare.dev계정(동시 요청 테스트용), 그중test01` 에 완료된 분석 25건(이력 페이지· 추이 확인용, 날짜 분산). 운영에서는 실행하지 않습니다.